

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0518

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

Real-Time Face Recognition System — OpenCV Edge Deployment

Objective

Develop a lightweight OpenCV system that detects and recognizes faces from a predefined dataset in real time on a Raspberry Pi.

Pipeline

Camera → Preprocessing → Face Detection → Feature Extraction → Recognition →

Output

1. Image Acquisition

- Capture live video using `cv2.VideoCapture(0)`.
- Resize frames to **640 × 480** for faster processing.

2. Preprocessing

- Convert frame to grayscale.
- Apply **CLAHE** to handle varying lighting conditions.

3. Face Detection

- Use **Haar Cascade Classifier**.
- Detect and crop the face region.

4. Feature Extraction & Recognition

- Resize face to a fixed size.
- Use **LBPH (Local Binary Pattern Histogram)** for recognition.
- Compare extracted features with the predefined face dataset.

OpenCV Code

```
import cv2

face_detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.read("face_model.yml")

cap = cv2.VideoCapture(0)

names = ["Unknown", "Anish", "Rahul", "Priya"]

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.resize(frame, (640, 480))
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
gray = clahe.apply(gray)

faces = face_detector.detectMultiScale(
    gray, scaleFactor=1.2, minNeighbors=5
)

for (x, y, w, h) in faces:
    face = gray[y:y+h, x:x+w]

    label, confidence = recognizer.predict(face)

    if confidence < 70:
        name = names[label]
    else:
        name = "Unknown"
    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.putText(frame, name, (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                (0, 255, 0), 2)
    cv2.imshow("Face Recognition", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()

```

Conclusion

The proposed OpenCV + Haar Cascade + CLAHE + LBPH pipeline is appropriate for low-power edge deployment because it minimizes computational and memory requirements while providing real-time face detection and recognition. CLAHE addresses illumination variation, while LBPH provides a lightweight recognition mechanism for a predefined dataset. Frame resizing and optional frame skipping further help maintain real-time performance on Raspberry Pi hardware.

6. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

Real-Time Object Detection on Mobile Device

Objective

Detect objects from a mobile camera in real time using OpenCV while minimizing CPU and memory usage.

Pipeline

Camera → Resize → Preprocessing → Object Detection → Bounding Box → Display

1. Image Acquisition

- Capture frames using the mobile camera.
- Resize frames to **320 × 240** to reduce computation.

2. Preprocessing

- Convert the frame to the required input format.
- Normalize pixel values.
- Process only required frames to reduce CPU usage.

3. Object Detection

- Use a lightweight model such as **MobileNet-SSD** with OpenCV DNN.
- Detect objects and obtain bounding boxes and confidence scores.

4. Display

- Draw bounding boxes and object names.
- Display the result on the mobile screen.

OpenCV Code

```
import cv2

net = cv2.dnn.readNetFromCaffe(

    "MobileNetSSD_deploy.prototxt",

    "MobileNetSSD_deploy.caffemodel"

)

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:

        break

    frame = cv2.resize(frame, (320, 240))

    blob = cv2.dnn.blobFromImage(

        frame, 0.007843, (300, 300), 127.5

    )

    net.setInput(blob)

    detections = net.forward()
```

```

for i in range(detections.shape[2]):

    confidence = detections[0, 0, i, 2]

    if confidence > 0.5:

        h, w = frame.shape[:2]

        box = detections[0, 0, i, 3:7] * [w, h, w, h]

        x1, y1, x2, y2 = box.astype("int")

        cv2.rectangle(frame, (x1, y1), (x2, y2),

                        (0, 255, 0), 2)

cv2.imshow("Object Detection", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

cap.release()

cv2.destroyAllWindows()

```

Efficiency Justification

- **320×240 resolution** → less CPU usage.
- **MobileNet-SSD** → lightweight detection.
- **Frame skipping** → improves real-time performance.
- **Small input size** → reduces memory consumption.
- **Confidence threshold** → avoids unnecessary detections.

Conclusion: The OpenCV DNN + MobileNet-SSD pipeline provides real-time object detection while satisfying the mobile device's CPU and memory constraints.

8. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

Motion Detection for Home Security

Objective

Detect significant movement from a camera feed while reducing false alarms caused by noise and background changes.

Pipeline

Camera → Preprocessing → Background Subtraction → Noise Removal → Motion Detection → Threshold → Alert

1. Image Acquisition

- Capture live video using OpenCV VideoCapture().
- Convert frames to grayscale.

2. Motion Detection

- Use **MOG2 Background Subtractor**.
- Compare the current frame with the background.

3. Noise Removal

- Apply **Gaussian Blur** to reduce camera noise.
- Apply **Morphological Opening** to remove small unwanted regions.
- Find contours and calculate their area.

4. Significant Movement

- Trigger an alert only when the contour area is greater than a predefined threshold, e.g. **500 pixels**.

OpenCV Code

```
import cv2

cap = cv2.VideoCapture(0)

bg = cv2.createBackgroundSubtractorMOG2()

while True:

    ret, frame = cap.read()

    if not ret:

        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    gray = cv2.GaussianBlur(gray, (5, 5), 0)

    mask = bg.apply(gray)

    contours, _ = cv2.findContours(

        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE

    )

    motion = False

    for c in contours:

        if cv2.contourArea(c) > 500:

            motion = True
```



```

x, y, w, h = cv2.boundingRect(c)

cv2.rectangle(frame, (x, y),
               (x+w, y+h), (0, 255, 0), 2)

if motion:

    cv2.putText(frame, "MOTION DETECTED",
                 (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
                 1, (0, 0, 255), 2)

cv2.imshow("Security Camera", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

cap.release()

cv2.destroyAllWindows()

```

Justification

- **MOG2** adapts to background variations.
- **Gaussian Blur** reduces image noise.
- **Contour-area threshold** ignores small movements such as leaves or noise.
- **Morphological filtering** can further remove small false regions.
- Alert is generated **only for significant movement**, reducing false alarms.

9. Document Scanner Application

A mobile app must:

- A. Capture document images
- B. Enhance readability
- C. Output clean scanned images

Design the OpenCV pipeline and justify preprocessing steps used for improving document quality.

Document Scanner Application

Objective

Capture a document using a mobile camera and convert it into a clean, readable scanned image.

Pipeline

Camera → Grayscale → Noise Removal → Edge Detection → Contour Detection → Perspective Correction → Thresholding → Output

OpenCV Code

```
import cv2

import numpy as np

img = cv2.imread("document.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

edges = cv2.Canny(blur, 50, 150)

contours, _ = cv2.findContours(

    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE

)
```

```
doc = max(contours, key=cv2.contourArea)

x, y, w, h = cv2.boundingRect(doc)

crop = gray[y:y+h, x:x+w]

scan = cv2.adaptiveThreshold(

    crop, 255,

    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,

    cv2.THRESH_BINARY, 11, 2

)

cv2.imwrite("scanned_document.jpg", scan)

cv2.imshow("Scanned Document", scan)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Justification

- **Grayscale:** reduces processing complexity.
- **Gaussian Blur:** removes camera noise.
- **Canny Edge Detection:** identifies document boundaries.
- **Contour Detection:** locates the document.
- **Perspective Correction:** removes tilted/angled appearance.
- **Adaptive Thresholding:** improves text readability under uneven lighting.

Conclusion: The pipeline produces a clean, high-contrast document image suitable for saving, sharing, or OCR.

12. Video Frame Quality Enhancement

A system must:

- A. Improve video quality in real time
- B. Reduce noise and enhance contrast

Design the pipeline and justify enhancement techniques used.

Video Frame Quality Enhancement

Objective

Enhance video frames in real time by reducing noise and improving contrast.

Pipeline

Camera → Frame Acquisition → Denoising → Contrast Enhancement → Output

OpenCV Code

```
import cv2

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:

        break

    # Noise reduction

    denoised = cv2.GaussianBlur(frame, (5, 5), 0)

    # Contrast enhancement

    lab = cv2.cvtColor(denoised, cv2.COLOR_BGR2LAB)

    l, a, b = cv2.split(lab)
```

```

clahe = cv2.createCLAHE(

    clipLimit=2.0, tileGridSize=(8, 8)

)

l = clahe.apply(l)

enhanced = cv2.cvtColor(

    cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR

)

cv2.imshow("Enhanced Video", enhanced)

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

cap.release()

cv2.destroyAllWindows()

```

Justification

- **Gaussian Blur:** reduces random noise and small artifacts.
- **LAB color space:** allows contrast enhancement without strongly affecting color.
- **CLAHE:** improves local contrast, especially in dark or unevenly illuminated regions.
- **Real-time processing:** simple OpenCV operations maintain fast frame processing.

Conclusion: The pipeline produces smoother and higher-contrast video frames while remaining suitable for real-time enhancement.